

PUC Minas

Pontifícia Universidade Católica de Minas Gerais

Curso de Engenharia de Software

Disciplina: Teste de Software

Análise de Eficácia de Testes com Teste de Mutação

Avaliação e melhoria da suíte de testes do projeto operacoes-mutante com StrykerJS e Jest

Aluno: Gabriel Cardoso dos Santos

Matrícula: 830691

Professor: Cleiton Tavares

Outubro de 2026

1. Introdução

Este relatório avalia, com teste de mutação, a suíte de testes do projeto *operacoes-mutante*: uma biblioteca de 50 funções matemáticas em JavaScript, testada com Jest. A ideia foi descobrir se a suíte realmente pega erros no código, entender por que os mutantes sobreviveram e escrever testes para matá-los.

Repositório: https://github.com/Cardosoooo/operacoes-mutante

Configuração: instalei `@stryker-mutator/core` e `@stryker-mutator/jest-runner` (versão 10.0) e rodei `npx stryker init`. No assistente, não usei preset, escolhi o Jest, mantive os reporters padrão (`html`, `clear-text` e `progress`), o npm e o formato JSON. Nesta versão o arquivo gerado se chama `stryker.config.json`; o `stryker.conf.json` do enunciado ainda é aceito, mas deixou de ser o nome padrão. A configuração usa `coverageAnalysis: "perTest"`, então cada mutante é testado só pelos testes que passam pelo trecho alterado. Também criei o script `npm run mutate` para rodar a análise.

2. Análise Inicial

Os 50 testes originais passam, e a cobertura do Jest (`npm test -- --coverage`) é alta: **98,64% das linhas** e 100% das funções, com 85,41% de statements e 58,82% de branches. A primeira execução do Stryker, porém, deu um mutation score de **73,71%** (Figura 1).

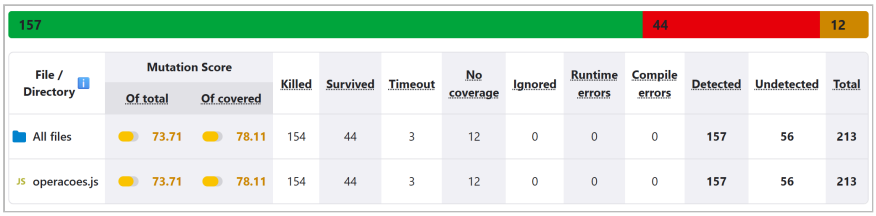


Figura 1. Primeira execução: 213 mutantes, 154 mortos, 3 por timeout, 44 sobreviventes e 12 sem cobertura.

Essa diferença acontece porque a cobertura só mostra se uma linha foi executada. Ela não diz se algum teste conferiu o resultado daquela linha. Todos os 44 sobreviventes estão em linhas cobertas: os testes passaram por elas, mas nenhum percebeu a alteração. Os números de statements e branches já davam uma pista. Cada função era chamada uma vez, quase sempre pelo caminho feliz, sem chegar aos `throw` e sem testar o lado falso das condições. Olhando os 56 mutantes não detectados um por um, separei as causas na Tabela 1.

Tabela 1. Causas dos 56 mutantes não detectados na primeira execução.

Fraqueza da suíte	Funções afetadas	Mutantes
Funções booleanas testadas só no caso `true`	`isPrimo` (9), `isPar`, `isImpar`, `isDivisivel`, `isMaiorQue`, `isMenorQue`, `isEqual`	18
Validações e arrays vazios nunca testados, ou `toThrow()` sem conferir a mensagem	`divisao`, `raizQuadrada`, `fatorial`, `mediaArray`, `maximoArray`, `minimoArray`, `inverso`, `medianaArray`	16
Entrada que não exige a lógica: array já ordenado e só de tamanho ímpar	`medianaArray`	9
Valores que anulam a operação (0 °C e 32 °F)	`celsiusParaFahrenheit`, `fahrenheitParaCelsius`	4
Valor sempre dentro do intervalo	`clamp`	4
Mutantes equivalentes (seção 5)	`fatorial`, `produtoArray`	5
Total		**56**

3. Análise de Mutantes Críticos

3.1 Conversão de temperatura: `celsius * 9/5` virou `celsius * 9 * 5`

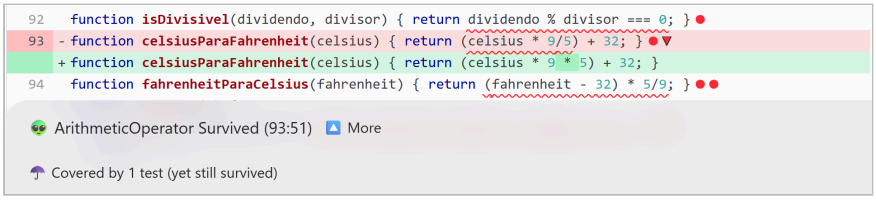


Figura 2. Mutante de `celsiusParaFahrenheit`, coberto pelo teste 38 e mesmo assim vivo.

O teste 38 converte 0 °C e espera 32 °F. Como o zero anula a multiplicação, `0 * 9/5`, `0 * 9 * 5` e `0 / 9 / 5` dão todos 0, e o resultado continua 32. A linha está coberta, mas o valor escolhido torna o fator irrelevante: qualquer

fórmula do tipo `celsius * k + 32` passaria. O teste 39 tem o mesmo problema: ele converte 32 °F, e como `32 - 32` dá 0, os dois mutantes de `fahrenheitParaCelsius` também sobrevivem.

3.2 Mediana: a ordenação foi removida



Figura 3. O Stryker removeu o `.sort()` e o teste 47 continuou passando.

O Stryker trocou `[...numeros].sort((a, b) => a - b)` por `[...numeros]`. O único teste da mediana usa `[1, 2, 3, 4, 5]`, que já está em ordem: ordenar ou não dá o mesmo array, e o elemento do meio continua 3. Por isso também sobreviveram as trocas do comparador por `() => undefined` e por `a + b`. Além disso, o array tem tamanho ímpar, então o ramo do array par nunca rodou: a linha 112 é a única descoberta no relatório de cobertura, e os 4 mutantes desse ramo ficaram sem cobertura. Ou seja, o teste usava justamente uma entrada em que a ordenação não faz diferença.

3.3 Clamp: `valor < min` virou `valor <= min`

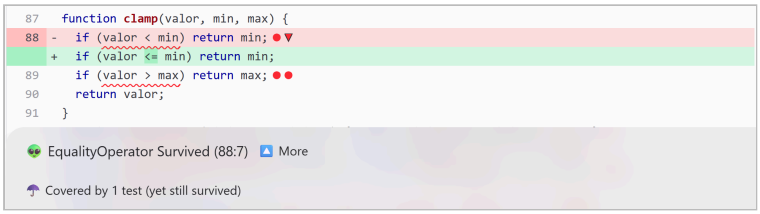


Figura 4. Mutante de fronteira em `clamp`, coberto pelo teste 36.

O teste 36 chama `clamp(5, 0, 10)`, um valor dentro do intervalo. As duas condições são avaliadas, por isso a linha aparece coberta, mas os `return min` e `return max` nunca rodam; trocar qualquer um dos `if` por `false` não muda nada. O mutante da figura é o mais sutil: nem o teste de fronteira que o exemplo do enunciado recomenda, com o valor igual ao limite (`clamp(0, 0, 10)`), o mata, porque quando `valor === min` tanto faz devolver `min` ou `valor`, o número é o mesmo. A única entrada que separa as duas versões é o zero com sinal. No original, `clamp(-0, 0, 10)` devolve o próprio `-0`, pois `-0 < 0` é falso; no mutante, `-0 <= 0` é verdadeiro e a função devolve o limite `0`. Como o `toBe` do Jest compara usando `Object.is`, que diferencia `0` de `-0`, dá para testar isso.

4. Solução Implementada

Mantive os 50 testes originais e adicionei 27 em um novo bloco, “Testes que matam os mutantes sobreviventes”, agrupados por função (`test/operacoes.test.js`, linhas 78 a 215). Cada teste foi escrito para falhar com um mutante específico, e todos passam no código original (`npm test`: 77 aprovados). A Tabela 2 mostra os que matam os mutantes da seção 3.

Tabela 2. Novos testes para os mutantes analisados e por que eles funcionam.

Mutantes	Novo teste	Por que mata
Fatores de `celsiusParaFahrenheit`	`celsiusParaFahrenheit(100)` → 212	Com 100 o fator importa: `* 9 * 5` dá 4532 e `/ 9 / 5` dá 34,2.
Fatores de `fahrenheitParaCelsius`	`fahrenheitParaCelsius(212)` → 100	`212 - 32 = 180`, e os mutantes dão 8100 e 4.
Ordenação da mediana	`medianaArray([1, 10, 2])` → 2	Fora de ordem: sem ordenar, o meio é 10. Também falharia com um `sort()` sem comparador, que ordena como texto.
Ramo do array par	`medianaArray([8, 2, 6, 4])` → 5	Faz a linha 112 rodar e confere a média dos dois do meio.
`if` do `clamp` trocados por `false`	`clamp(-5, 0, 10)` → 0 `clamp(15, 0, 10)` → 10	Fazem os `return min` e `return max` rodarem.
`<` → `<=` e `>` → `>=` no `clamp`	`clamp(-0, 0, 10)` → -0 `clamp(0, -10, -0)` → 0	Único caso em que devolver o limite é diferente de devolver o valor.

Os outros testes seguem a mesma ideia e se dividem em três grupos. **O caso falso das funções booleanas**, incluindo a igualdade, que separa `>` de `>=`: `isMaiorQue(5, 5)`, `isMenorQue(5, 5)`, `isPar(7)`, `isImpar(4)`, `isDivisivel(10, 3)` e `isEqual(7, 8)`; no `isPrimo`, os compostos 4 e 9 obrigam o laço a achar um divisor, e o 1 separa `n <= 1` de `n < 1`. **As validações**: a entrada inválida com `toThrow` e a mensagem esperada, o que também mata a troca da mensagem por texto vazio, e o valor válido na fronteira, como `raizQuadrada(0)` e `fatorial(0)`, que mata a troca de `n < 0` por `n <= 0`. **Os casos vazios**, como `mediaArray([])` → 0: sem o `if`, a função faria `0 / 0` e devolveria `NaN`.

Considero esses testes eficazes porque cada um foi escrito pensando em um mutante específico e conferido rodando o Stryker de novo. Eles também verificam só o resultado da função (o valor ou o erro lançado), sem depender de como ela foi implementada. Durante a análise, percebi ainda que as mensagens de `maximoArray`, `minimoArray` e `medianaArray` têm "не" (em cirílico) no lugar de "não". Como o foco do trabalho são os testes, não mexi no código: os testes conferem só o final da mensagem, e deixo o problema registrado aqui.

5. Resultados Finais

Tabela 3. Evolução das execuções do Stryker (213 mutantes em todas).

Execução	Score	Mortos	Timeout	Sobreviventes	Sem cobertura	Ignorados
Suíte original (50 testes)	**73,71%**	154	3	44	12	0
Com os 27 testes novos	97,65%	205	3	5	0	0
Final, com os equivalentes marcados	**100,00%**	203	3	0	0	7

Com os novos testes, 208 dos 213 mutantes passaram a ser detectados. Os 5 que sobraram são **equivalentes**: eles alteram o código, mas a função continua devolvendo os mesmos resultados, então nenhum teste consegue matá-los.

- **`fatorial`, linha 19 do código original (4 mutantes):** o `if (n === 0 || n === 1) return 1` trocado por `false`, por `&&` ou com um dos lados falso. Para 0 e 1, o laço `for (let i = 2; i <= n; i++)` não executa e a função devolve `resultado = 1`, o mesmo valor do retorno antecipado.
- **`produtoArray`, linha 84 (1 mutante):** sem o `if`, o `reduce` com valor inicial 1 também devolve 1 para o array vazio.

Para esses casos, usei o comentário `// Stryker disable next-line <mutador>: motivo`, que é a forma que o próprio Stryker oferece para marcar mutantes equivalentes. Eles passam a aparecer como *Ignored* e não entram no cálculo; o código continua funcionando igual. O comentário vale para todos os mutantes daquele tipo na linha, então acabou ignorando também 2 que já estavam mortos (as trocas da condição por `true`). Por isso aparecem 7 ignorados. Os 3 timeouts são laços infinitos (`i++` virando `i--` no `fatorial` e duas mutações no `while` do `mdc`), e o Stryker conta esses casos como detectados.

No final, o score ficou em **100%** (206 de 206, Figura 5). Sem os comentários ele seria 97,65%, e os 5 mutantes que faltam são justamente os equivalentes. A cobertura também chegou a 100% nas quatro métricas. Esses mutantes mostram ainda que os dois `if` são redundantes e poderiam ser apagados, mas não mexi neles porque o trabalho é sobre a suíte de testes.

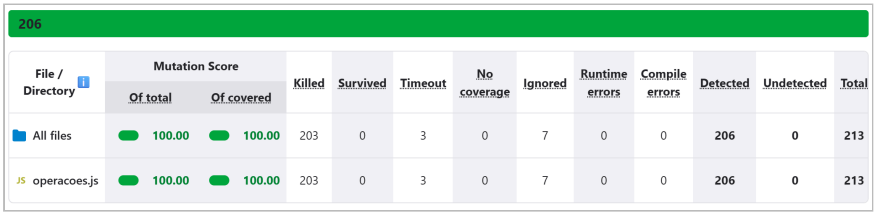


Figura 5. Execução final: 203 mortos, 3 por timeout, 7 ignorados e nenhum sobrevivente.

6. Conclusão

Antes deste trabalho, eu olharia para 98,64% de cobertura e acharia que a suíte estava boa. O teste de mutação mostrou que não estava: os testes passavam pelo código, mas muitos não conferiam quase nada. Eles testavam um caso só por função, usavam valores como 0 °C, que escondem erros na fórmula, e testavam a mediana com um array que já estava ordenado. O que achei mais útil é que cada mutante sobrevivente mostra exatamente qual teste está faltando. Por outro lado, a ferramenta não resolve tudo sozinha: precisei analisar caso a caso para saber quais mutantes eram equivalentes, e no `clamp` um que parecia equivalente não era. Mesmo assim, para saber se os testes pegariam um erro de verdade no código, o teste de mutação diz muito mais do que a cobertura.